

1. Introduction

1.1 Problem Description

This Software Requirements Specification (SRS) document provides a complete description of the Student Dropout Prediction System using AI. The document is intended for:

1. Project supervisors and academic evaluators.
2. System administrators (college IT staff).
3. Faculty mentors and department heads.
4. Quality assurance testers.

1.2 Scope

Software Product Name: Student Dropout Prediction System with AI-Powered WhatsApp Chatbot.

What the system WILL do:

- **Shall** collect both academic (marks, attendance, credits) and behavioral (sentiment, stress indicators, engagement) data
- **Shall** utilize LangChain framework with Google Gemini API for natural language conversations via WhatsApp
- **Shall** predict dropout probability using Random Forest classification (target: $\geq 80\%$ accuracy)
- **Shall** automatically connect at-risk students with faculty mentors
- **Shall** provide real-time monitoring dashboards for administrators and mentors
- **Shall not** replace academic counseling but augment early detection capabilities

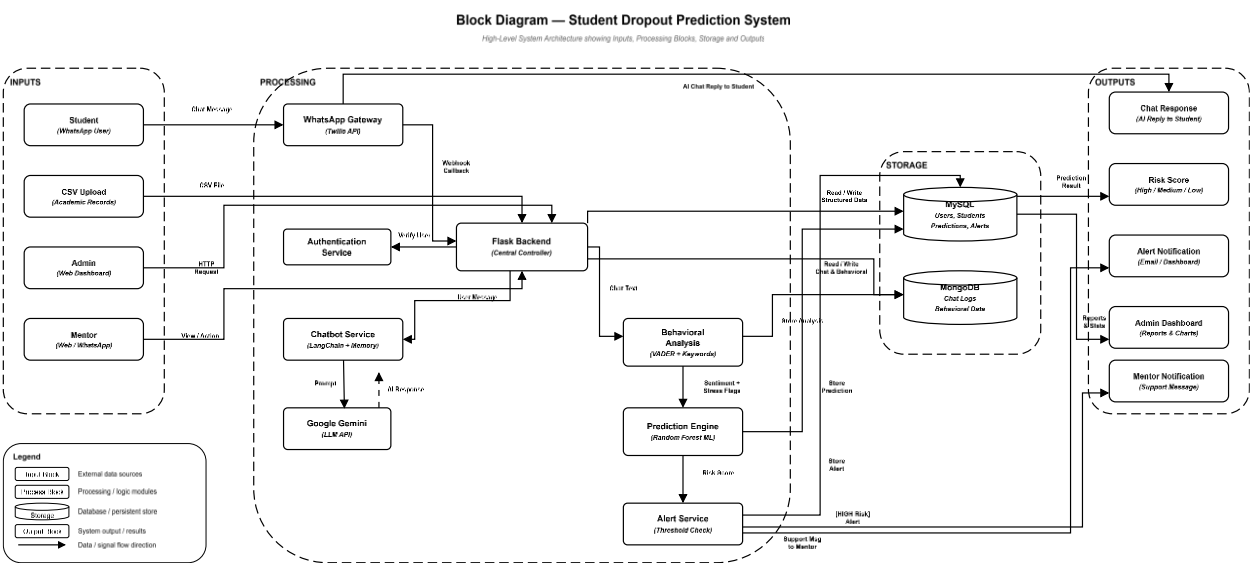
Application Benefits:

- **For Students:** Anonymous, judgment-free emotional support and early academic intervention
- **For Faculty:** Data-driven insights into student well-being and automated risk alerts
- **For Institution:** Reduced dropout rates, improved retention metrics, and systematic mentoring workflows

2. Overall description

2.1 Product perspective

2.1.1 Block Diagram:



2.1.2 Hardware Requirements:

Component	Specification
Processor	Intel Core i5 or above
RAM	8 GB (16 GB recommended)
Storage	256 GB SSD
Internet	10 Mbps broadband connection
Mobile Device	Android/iOS smartphone with WhatsApp installed

2.1.3. Software Requirements:

Category	Technology	Version/Purpose
Operating System	Ubuntu Server / Windows Server / Cloud (AWS/Azure)	Deployment environment
Backend Runtime	Python	3.10+
Web Framework	Flask	REST API development
AI/ML Framework	LangChain, Scikit-learn	Chatbot orchestration and prediction
LLM Service	Google Gemini API	Conversational AI (Free tier)
Frontend	React.js	Admin and Mentor portals
Real-time Communication	Socket.io	Mentor-student chat
Databases	MySQL 8.0, MongoDB 6.0	Structured and unstructured data

2.2 Product functions

Major Functional Capabilities:

1.Student Data Collection

- Upload student records via CSV
- Manual data entry through admin panel
- Validate and store student information

2.WhatsApp Chatbot Interaction

- Natural language conversations with students
- Weekly check-ins and wellness queries
- Context-aware responses using conversation memory
- Behavioral data extraction from conversations

3. Behavioral Analysis

- Sentiment analysis of student messages
- Stress level detection
- Engagement score calculation
- Dropout signal identification

4. Machine Learning Prediction

- Combine academic and behavioral features
- Predict dropout probability (0-100%)
- Classify risk levels (LOW/MEDIUM/HIGH)
- Update predictions based on new data

5. Mentor Assignment & Communication

- Automatic mentor assignment for high-risk students
- Real-time chat between students and mentors
- Chat history storage and tracking

6. Admin Dashboard

- Overview of all students with risk scores
- Risk distribution visualization
- Alert management
- Mentor management and workload monitoring

7. Mentor Portal

- View assigned students
- Monitor student risk trends
- Chat with students
- Add notes and track follow-ups

3. SPECIFIC REQUIREMENTS

3.1 External Interfaces

- **Inputs:**

- **Student Data:** Academic records (marks, attendance, credits) uploaded via CSV or manual entry.
- **User Interactions:** Natural language student messages received through the WhatsApp chatbot interface.
- **Mentor/Admin Input:** Mentor notes, follow-up tracking, and administrative manual data entry.

- **Outputs:**

- **Predictions:** Dropout probability scores (0-100%) and risk level classifications (Low/Medium/High).
- **Automated Responses:** Context-aware, AI-generated WhatsApp messages to students.
- **Alerts & Visualizations:** Real-time risk distribution dashboards and automated risk alerts for faculty mentors.
- **Communications:** Real-time chat messages between students and mentors.

3.2 Functional Requirements

3.2.1 ML Prediction Module

The system shall classify risk levels as:

- **LOW:** 0-30% (Action: Monitor)
- **MEDIUM:** 31-70% (Action: Watch)
- **HIGH:** 71-100% (Action: Assign Mentor)

The system shall achieve a minimum prediction accuracy of 80%.

3.2.2 Mentor-Student Chat Module

The system shall provide real-time chat functionality between students and assigned mentors using Socket.io.

The system shall authenticate users (student and mentor) before granting chat access.

The system shall display assigned mentors to students upon login.

The system shall persist chat history in MongoDB for future reference and analysis.

The system shall support text-based messaging with typing indicators.

3.2.3 Admin Panel Module

The system shall provide a dashboard overview showing total students, risk distribution (pie chart), and recent alerts.

The system shall display a sortable student list with risk scores and categories.

The system shall display detailed student profiles including academic info, behavioral summary, prediction details, and assigned mentor.

The system shall allow administrators to add/remove mentors, view mentor workloads, and assign mentors to students.

The system shall generate alerts for high-risk students and crisis situations, with status tracking (resolved/unresolved).

3.2.4 Mentor Portal Module

The system shall display a list of assigned students with risk scores and levels.

The system shall provide quick view of student concerns and behavioral trends.

The system shall allow mentors to view chat summaries and initiate direct chat with students.

The system shall provide functionality for mentors to add notes about students, track follow-ups, and record meeting outcomes.

3.3 Performance Requirements

Accuracy: The ML prediction model shall achieve a minimum accuracy of 80% for dropout risk classification.

Response Time:

- WhatsApp chatbot responses shall be generated within 3 seconds.
- Admin dashboard queries shall load within 2 seconds.
- Mentor-student chat latency shall not exceed 500ms.

Throughput: The system shall support a minimum of 100 concurrent users (students chatting simultaneously).

Availability: The system shall maintain 99% uptime during academic hours (8 AM - 8 PM).

Data Processing: The system shall process and analyze behavioral data from chat logs within 1 hour of conversation completion.

3.4 Design Constraints

Cost Constraint: The system shall utilize free tiers of external APIs (Google Gemini API free tier, Twilio Sandbox for WhatsApp) to ensure zero operational cost for pilot deployment.

Privacy Compliance: The system shall comply with educational data privacy standards (FERPA in US context, or equivalent institutional data protection policies). Personal identifiers shall be encrypted at rest.

Platform Limitation: WhatsApp integration is limited to Twilio's Sandbox environment for development, requiring students to join specific sandbox sessions.

Browser Compatibility: Web portals shall support Chrome, Firefox, Safari, and Edge (latest two versions).

3.5 Logical Database Requirements

3.5.1 Data Entities and Relationships

- **Student Entity:** (Roll No, Name, Gender, Age, Academic Marks, Risk Score, Risk Level, Assigned Mentor ID)
- **Mentor Entity:** (Mentor ID, Name, Department, Email, Workload)
- **Conversation Entity:** (Message ID, Student ID, Timestamp, Message Content, Sentiment Score, Stress Count)
- **Behavioral Features Entity:** (Student ID, Date, Avg Sentiment, Message Frequency, Engagement Score, Dropout Signal Flag)
- **Chat History Entity:** (Chat ID, Student ID, Mentor ID, Message, Timestamp, Sender Type)

3.5.2 Data Retention

- **Academic records:** Retained for 4 years (program duration + 1 year)
- **Chat logs:** Retained for 1 year post-student graduation/withdrawal
- **Behavioral analysis data:** Aggregated monthly, raw data purged after 6 months

3.5.3 Integrity Constraints

- Student Roll No shall be unique and non-null
- Risk scores shall be constrained between 0-100
- Foreign key constraints between Student-Mentor assignments
- Timestamp automatic generation for all chat records

3.6 Software System Attributes

3.6.1 Reliability

The system shall implement automated backup of MySQL databases every 24 hours. In case of ML service failure, the system shall default to rule-based risk assessment using academic thresholds (marks < 40% = HIGH risk).

3.6.2 Availability

The system shall implement health checks for external APIs (Gemini, Twilio). If WhatsApp service is unavailable, the system shall queue messages and retry every 5 minutes for up to 1 hour.

3.6.3 Security

- Authentication: JWT (JSON Web Token) based authentication for Admin and Mentor portals
- Authorization: Role-based access control (RBAC) with three roles: Admin, Mentor, Student
- Data Protection: AES-256 encryption for sensitive student data in database
- API Security: Rate limiting on Flask API (100 requests/minute per IP)
- Input Validation: SQL injection and XSS prevention on all input fields

3.6.4 Maintainability

The system shall follow modular architecture allowing independent updates to:

- ML model (pluggable scikit-learn models)
- LLM provider (swappable between Gemini, OpenAI, or local LLMs)
- Database backend (MySQL/MongoDB abstraction layers)

3.6.5 Portability

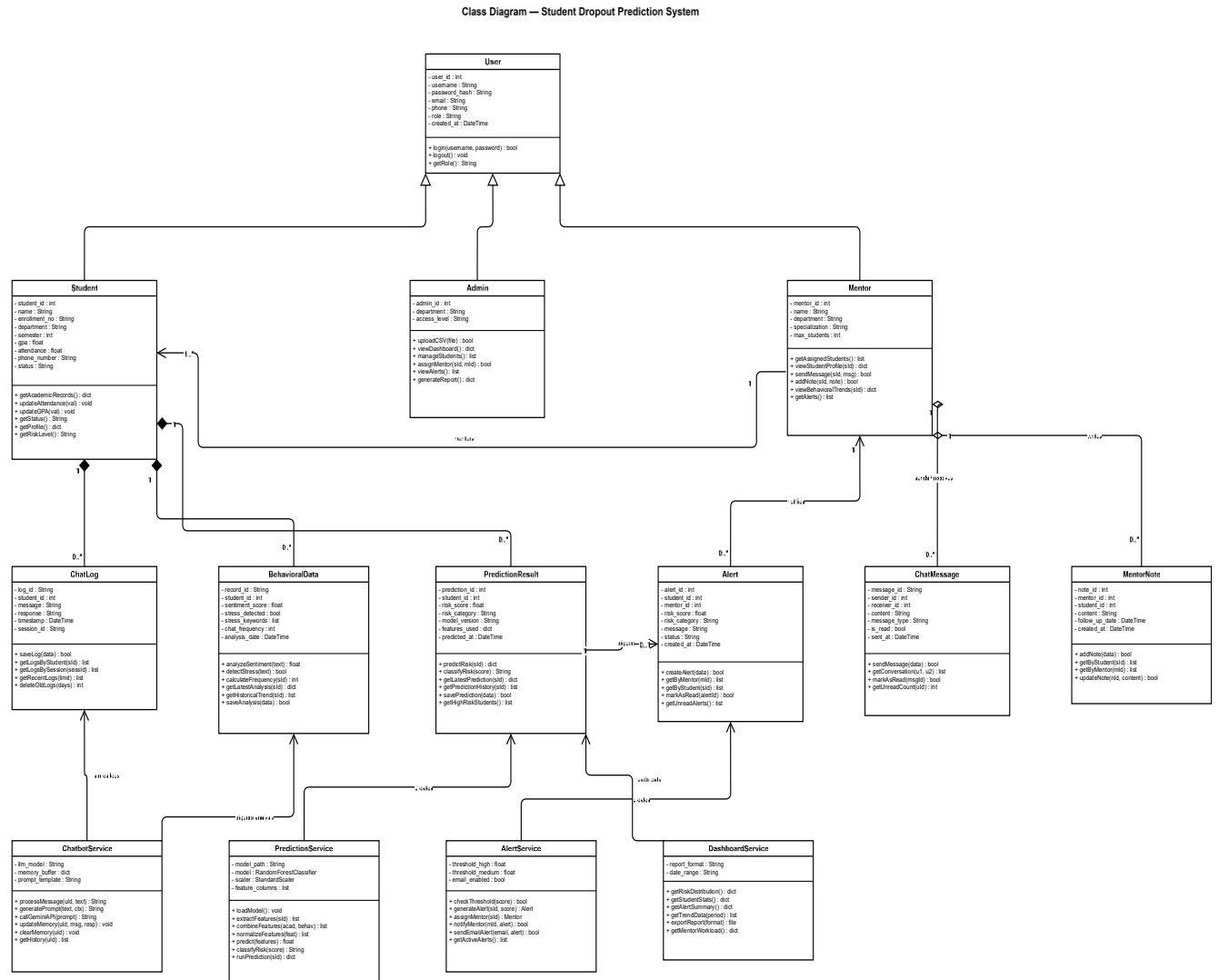
The system shall be containerized using Docker to ensure deployment across:

- Local development environments (Windows/Linux/Mac)
- Cloud platforms (AWS, Azure, Google Cloud)
- On-premise college servers

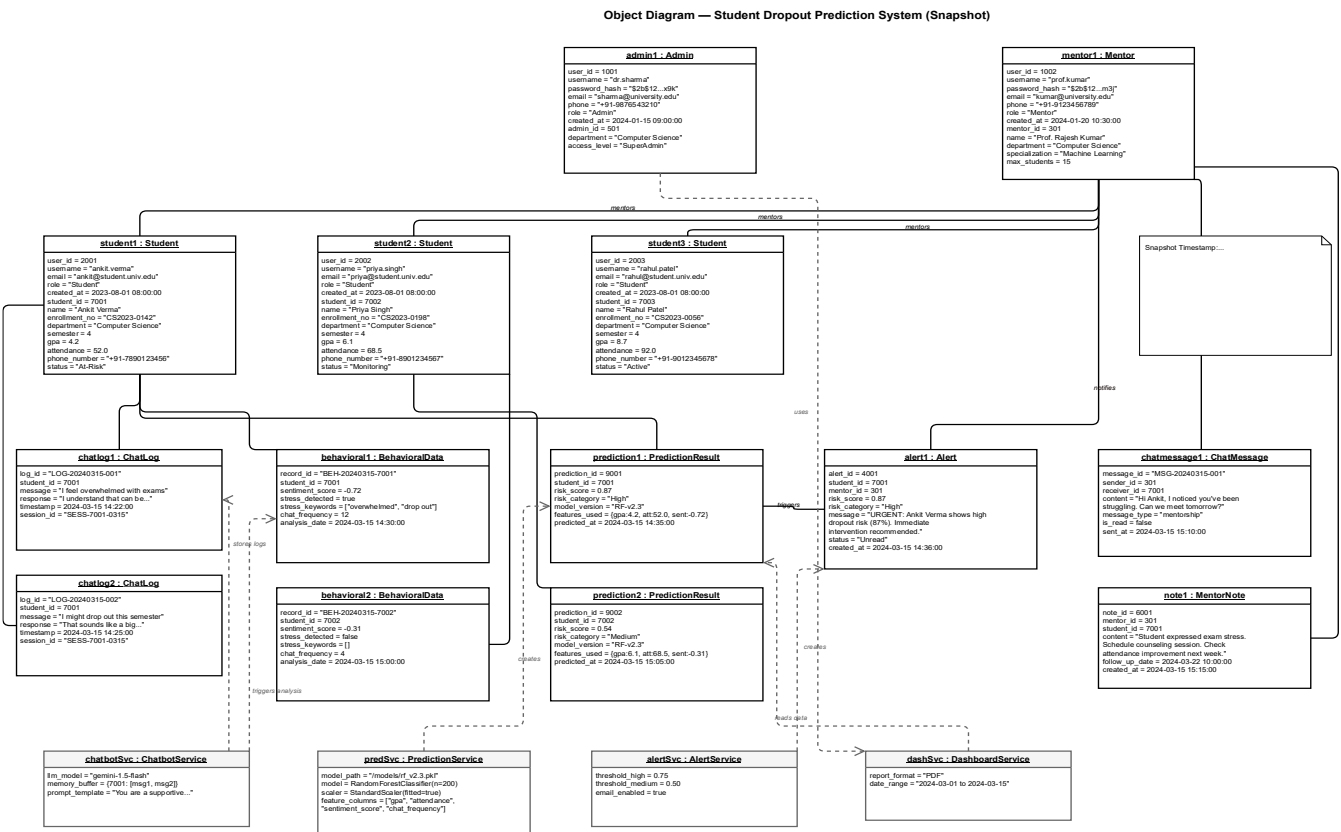
4. SOFTWARE DESIGN DOCUMENT

4.1 Structural Design

4.1.1 Class Diagrams:

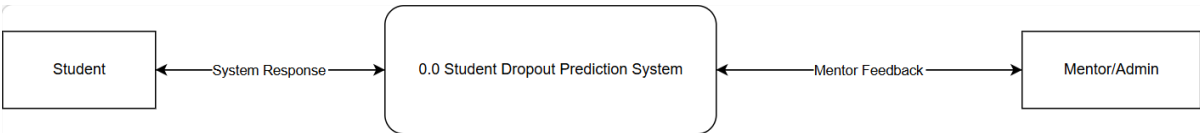


4.1.2 Object Diagrams:.

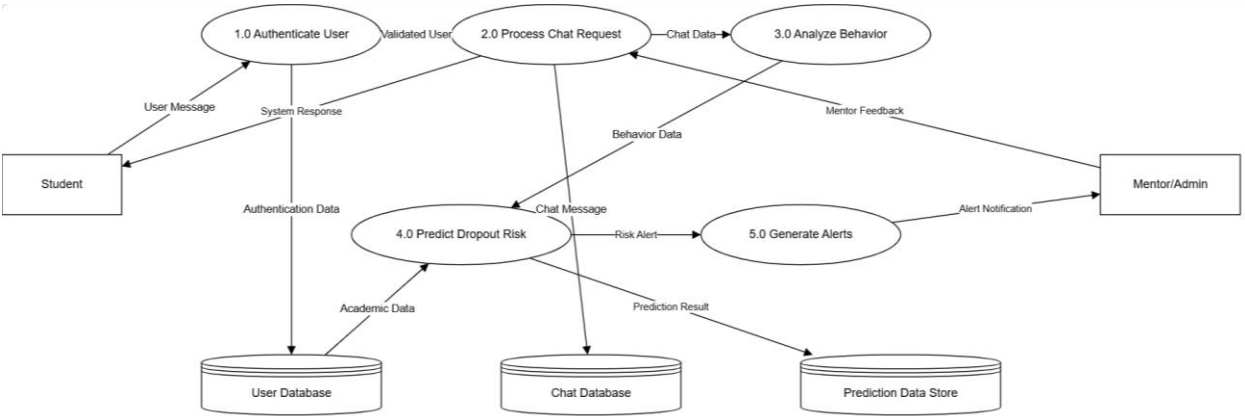


4.2 Data Design

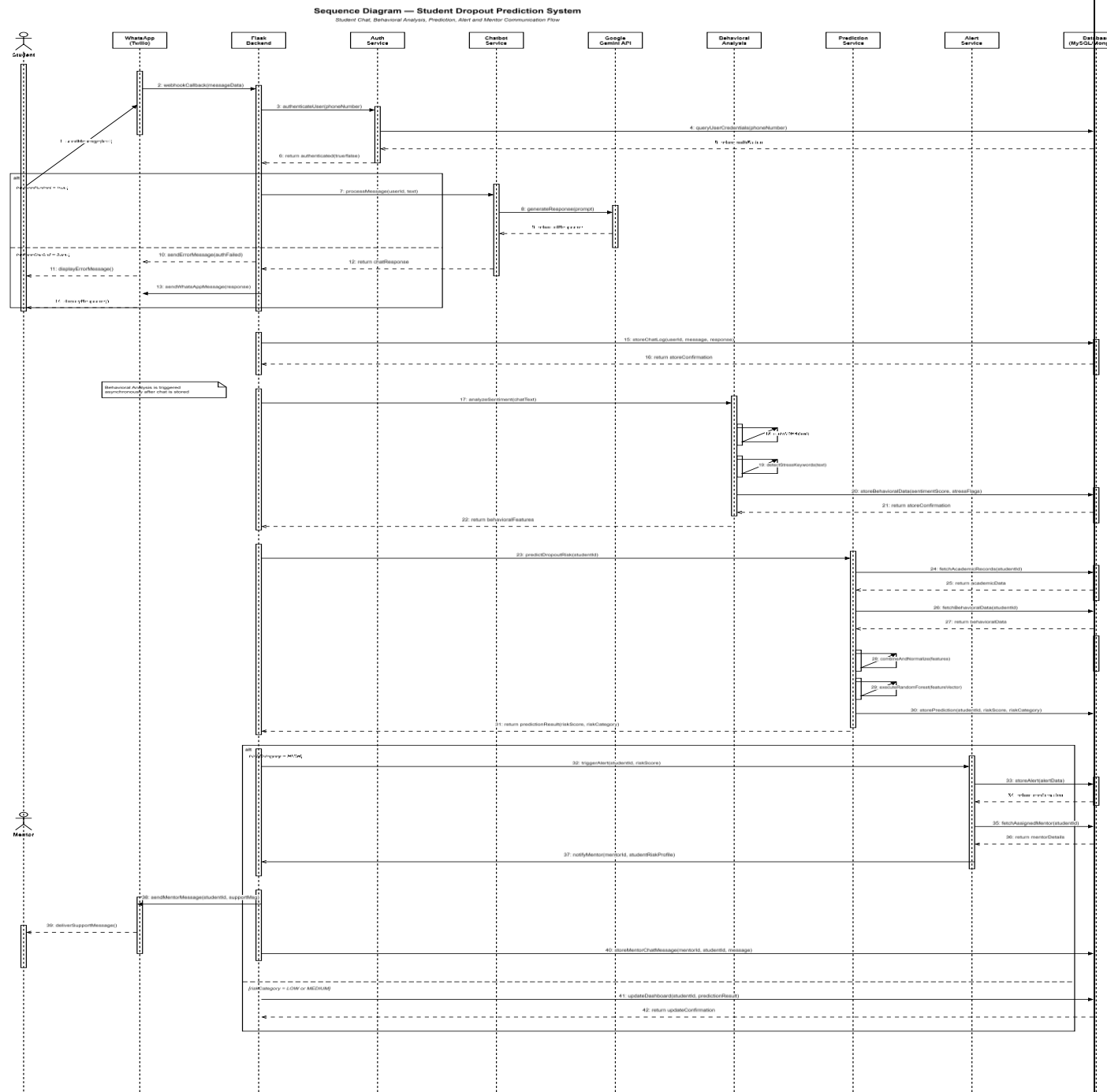
DFD0:



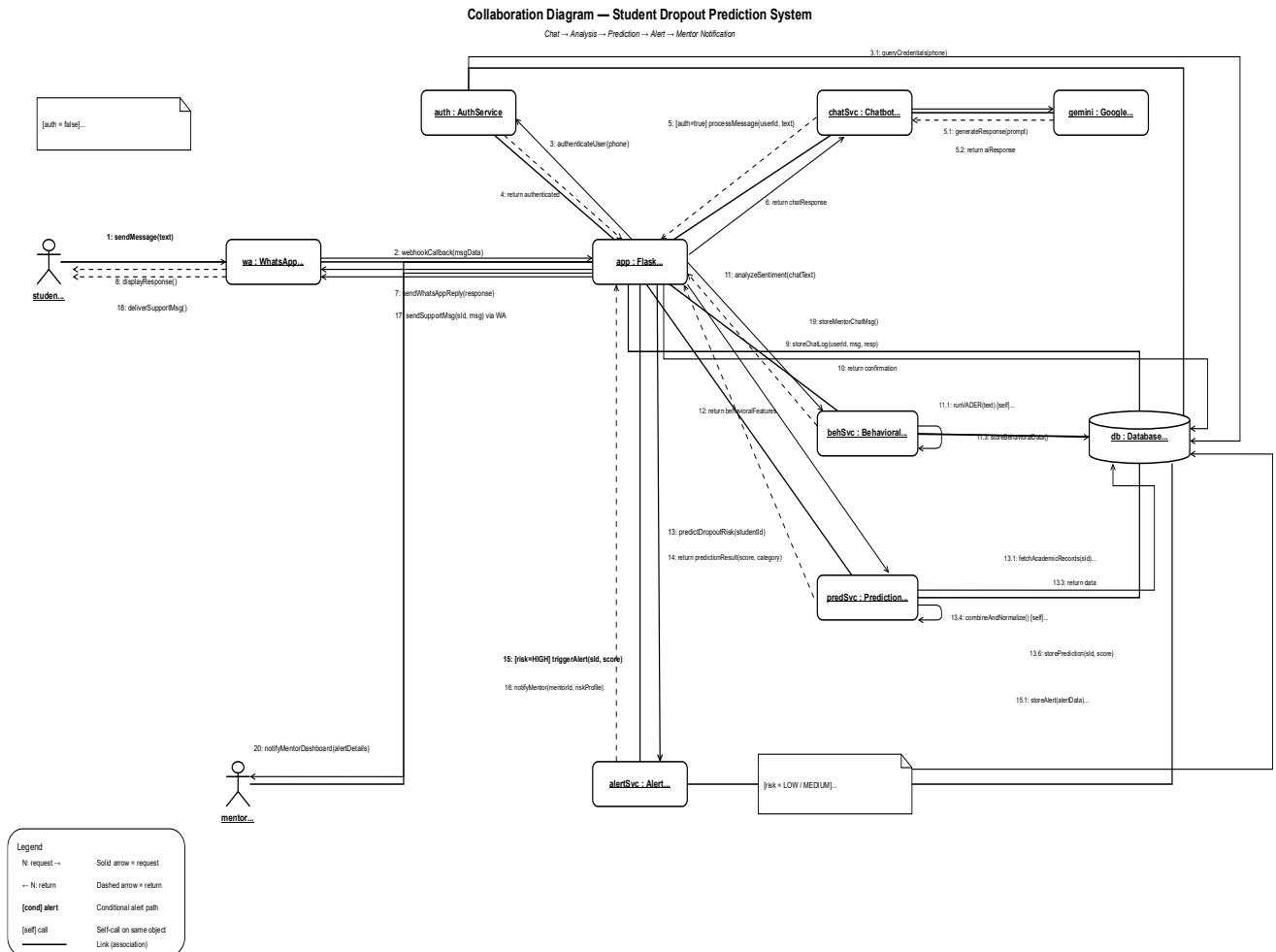
DFD1:



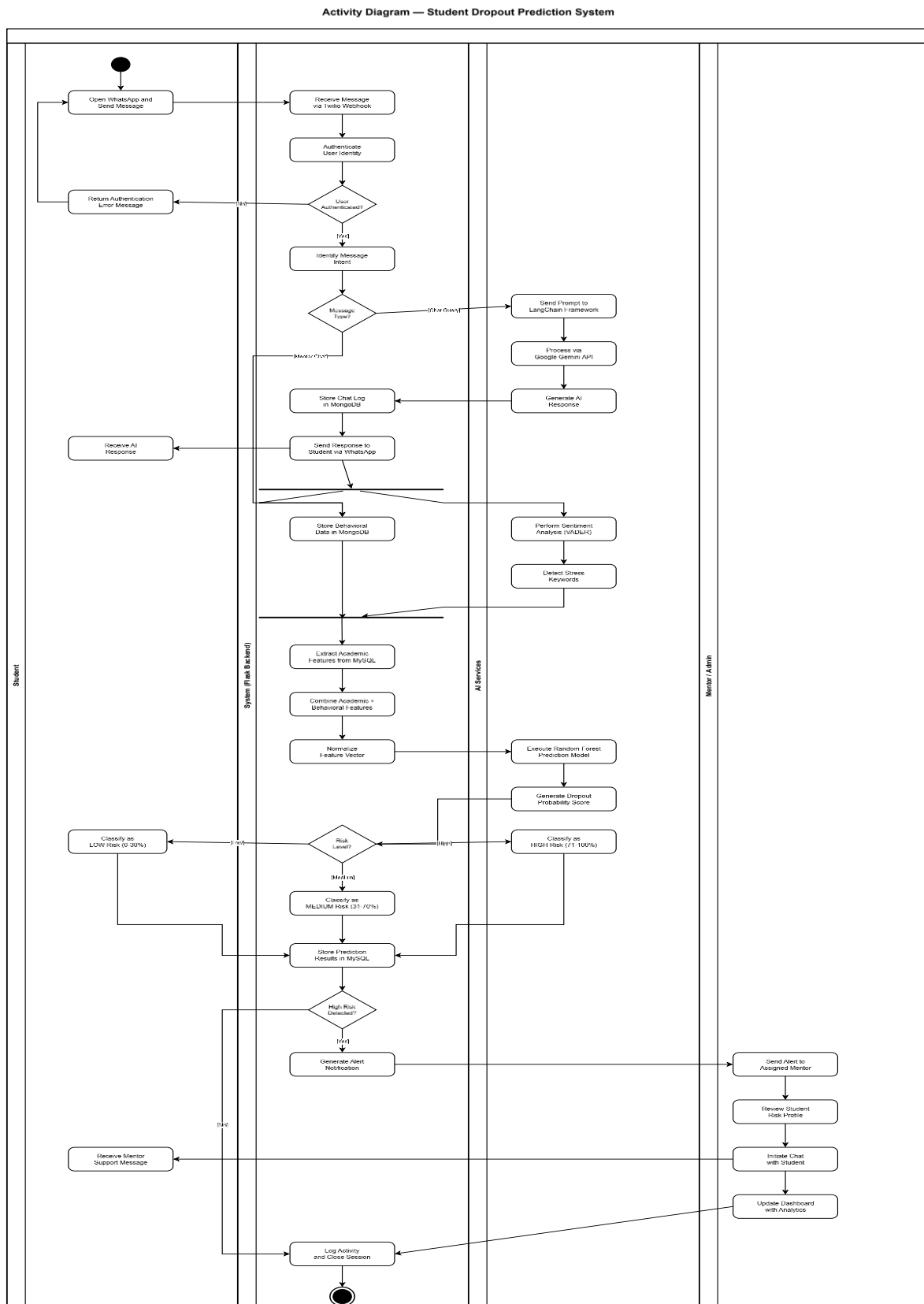
4.3.1 Sequence Diagram:



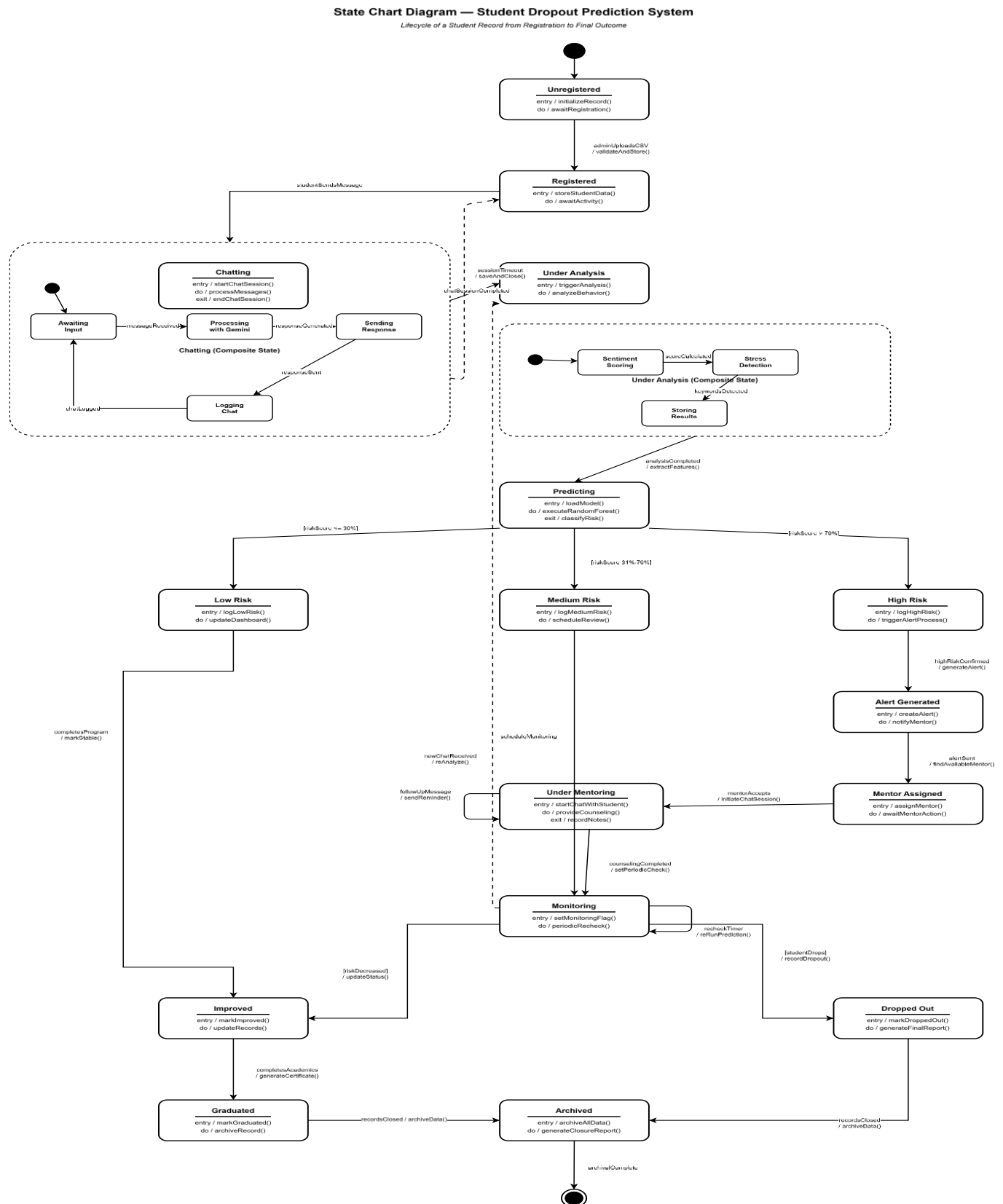
4.3.2. Collaboration diagram



4.3.3 Activity Diagram:

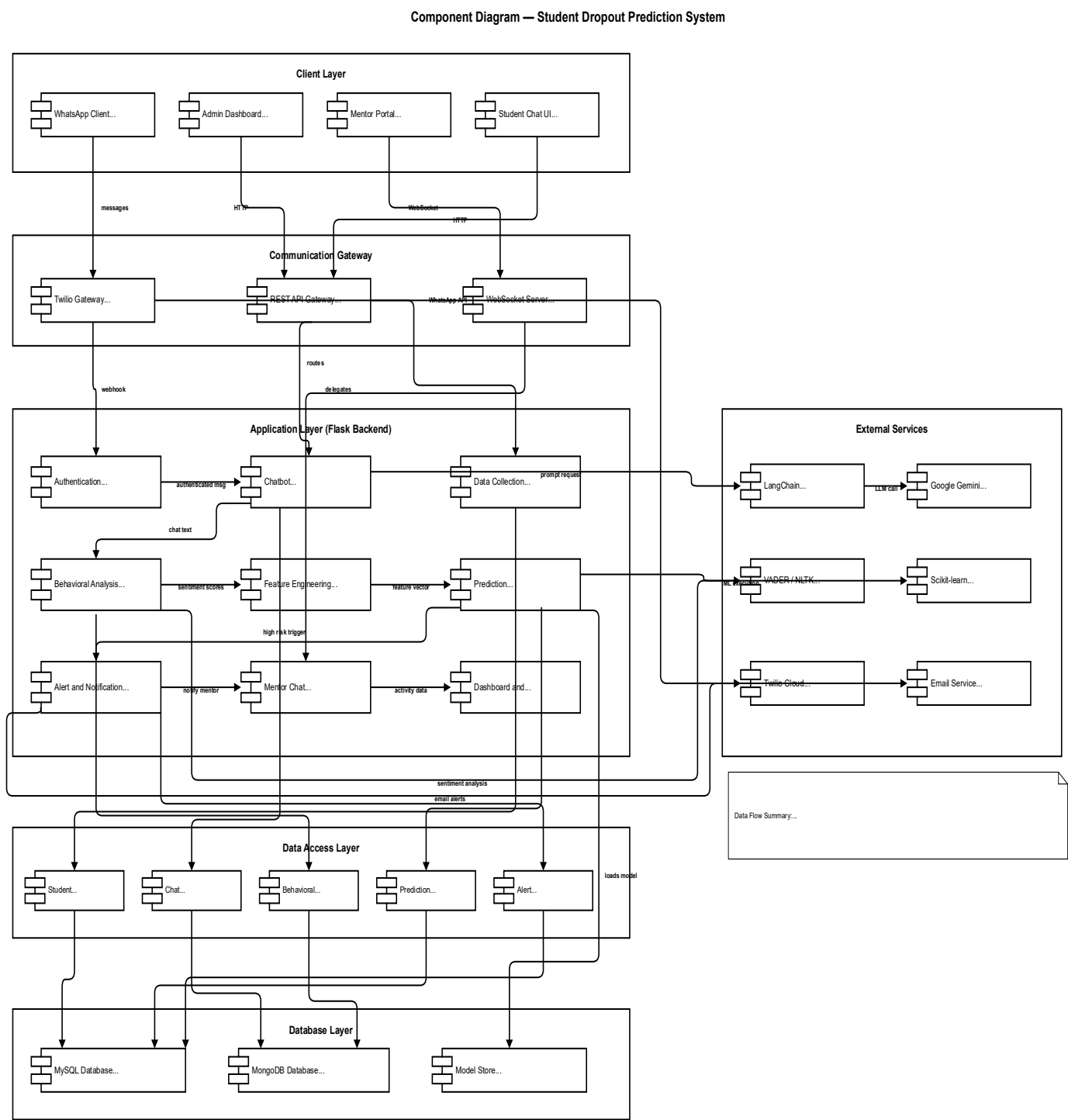


4.3.4 State-chart Diagram:

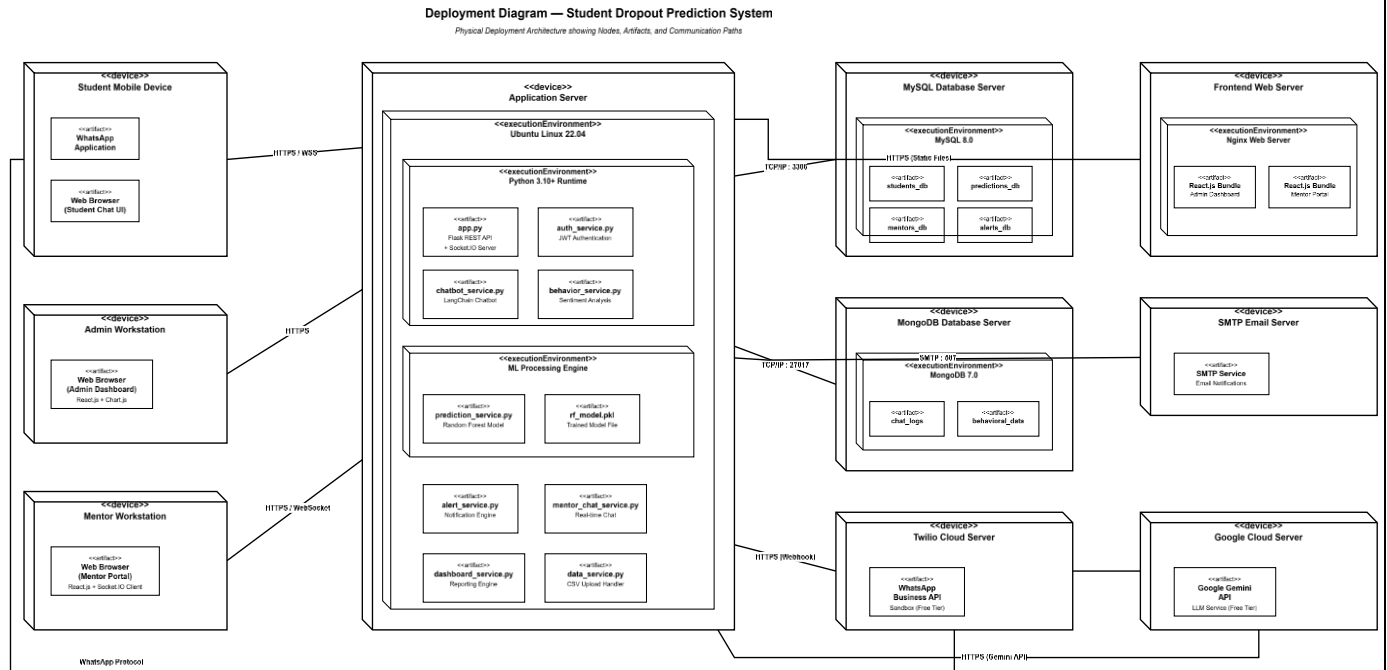


4.4 Implementation Design

4.4.1 Component Diagram:



4.4.2 Deployment Diagram:



5. REFERENCES

- [1] F. Del Bonifro, M. Gabrielli, G. Lisanti, and S. P. Zingaro, "Student Dropout Prediction," in *Artificial Intelligence in Education*, Springer, 2020, pp. 129-140.
- [2] C. J. Hutto and E. Gilbert, "VADER: A Parsimonious Rule-Based Model for Sentiment Analysis," *ICWSM*, 2014.
- [3] LangChain Documentation, <https://python.langchain.com/docs/>
- [4] Google Gemini API Documentation, <https://ai.google.dev/docs>
- [5] Twilio WhatsApp API Documentation, <https://www.twilio.com/docs/whatsapp>
- [6] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *JMLR*, vol. 12, 2011.
- [7] IEEE Standard 830-1998, *IEEE Recommended Practice for Software Requirements Specifications*, IEEE, 1998.
- [8] D. Andrade-Girón et al., "Predicting Student Dropout using ML and DL: A Systematic Review," *EAI Transactions*, 2023.

